Does Microsoft provide the model-provider layer?

Research brief | Microsoft Foundry evaluation frameworks | 8 September 2026

Prepared for the InspectAzureAI .NET project. This report treats a provider layer as the adapter that translates a shared application interface into a model's protocol, messages, tools, options and responses. It distinguishes the model being tested from the model judging its output.

Yes - the closest .NET match is IChatClient

Microsoft supplies the shared `Microsoft.Extensions.AI.IChatClient` interface, and its evaluation libraries build on the Microsoft.Extensions.AI abstractions. Microsoft Agent Framework also integrates model clients and evaluation. This is a close architectural equivalent to the project's provider contract. It lets evaluation code consume a common representation while provider adapters handle the underlying service. [1, 2, 3]

The qualification is coverage: a shared interface does not make every Foundry deployment support every task or option. There are separate adapters for Chat Completions, Responses and native Anthropic Messages, plus platform-side model invocation in Foundry cloud evaluation. Their supported features differ. [4, 5, 6]

Microsoft offering	What it supplies	Fit for this project
Microsoft.Extensions.AI	Common IChatClient interface, messages, options and composable middleware	Closest equivalent to the model-provider boundary. [1]
Microsoft.Extensions.AI.Evaluation	Quality, safety and deterministic evaluators; reporting and caching	Reuse scorers and reporting alongside an existing test runner. [2]
Microsoft Agent Framework	Provider-backed agents and built-in agent evaluation	Useful when evaluating complete agent interactions and tool use. [3]
Foundry cloud evaluation	Service-managed evaluation with model and agent targets	An alternative execution location for supported targets. [7]
Azure AI Evaluation SDK for Python	Dataset evaluation with a target callable or saved responses	Application code supplies the callable's model integration. [8]

Recommendation: use Microsoft's interfaces and supported adapters where they reduce maintenance. Retain the comparison runner and deployment-specific behavior until equivalent behavior has been demonstrated. Your repository already exposes an Inspect model through `InspectChatClient`, so there is an existing integration point. [14]

How the communication layer works

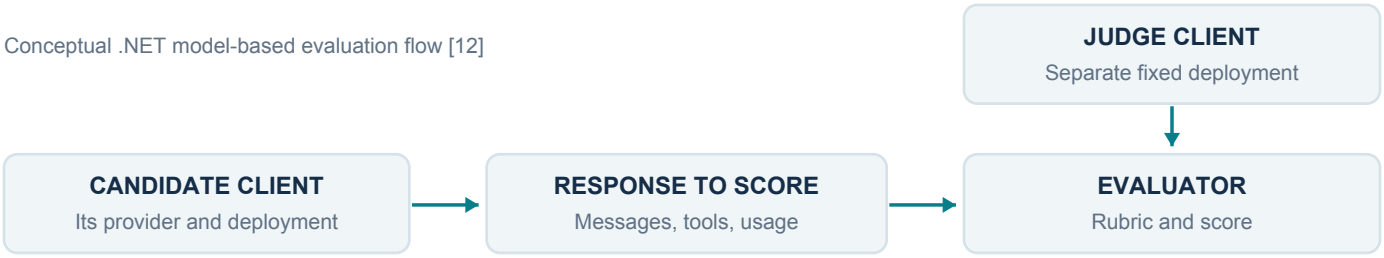
One application interface, several transports

Communication path	Documented .NET integration	Boundary to preserve
OpenAI-compatible Chat Completions	ChatClient becomes an IChatClient through Microsoft.Extensions.AI.OpenAI	Choose a deployment that supports this route and the requested options. [4]
OpenAI Responses	ResponsesClient has a separate IChatClient adapter; Foundry also exposes project Responses	State, hosted tools and response items differ from Chat Completions. The inspected adapter overload is marked experimental. [4, 6]
Native Anthropic Messages on Foundry	AnthropicFoundryClient uses Anthropic's IChatClient integration; Microsoft's package wraps it as an agent	Anthropic supplies the native SDK adapter. Microsoft supplies the common abstraction and Agent Framework integration. [5, 9, 10]
Admin-connected model in cloud evaluation	A connection/deployment reference lets Foundry resolve endpoint and authentication	Preview support requires an OpenAI Chat Completions endpoint at the connected deployment. [11]

The Foundry SDK also offers a project endpoint for model access, agents and evaluations. Its broad catalog-access language should be read with its endpoint guidance: native Claude is documented separately through Anthropic Messages. This report does not infer universal native-protocol compatibility from the phrase "OpenAI-compatible client." [6]

The candidate and judge use separate connections

Conceptual .NET model-based evaluation flow [12]



The .NET evaluator contract receives the conversation and the response to score. A separate `ChatConfiguration` supplies the judge's `IChatClient` for model-based scoring. A Claude candidate can therefore be judged by a separately configured GPT client. Deterministic evaluators need no judge model; safety evaluators can use the Foundry evaluation service. [12, 2]

For your scoreboard: keep one judge configuration fixed across candidate runs. Matching the provider interface solves transport interoperability; it does not establish that a judge is accurate on the expense task. Preserve task-specific checks for required facts, tool use and successful completion.

What this means for your code

Start from the bridge that already exists

The repository's `InspectAzureAI.Maf/InspectChatClient.cs` already implements `IChatClient`. It converts Microsoft messages, tools and options, calls `AgentBridge.GenerateAsync`, and converts the result to `ChatResponse`. The existing Inspect model remains responsible for generation. This connects Microsoft consumers to your providers; it does not replace your providers with Microsoft's transports. [14]

The same file implements streaming by first waiting for the complete response and then yielding converted updates. That is adequate for some batch evaluations, but it cannot establish real time-to-first-token behavior. The project references `Microsoft.Agents.AI` version 1.20.0; current web documentation and repository main branches must be checked against installed packages before copying examples. [14]

A practical adoption path

1. **Reuse the common boundary.** Feed the existing bridge's responses to selected Microsoft evaluation components, with a separate judge client. This is an architectural recommendation based on the two contracts; this research did not execute that integration. [12, 14]

2. **Compare one native adapter per protocol.** Try a compatible Chat Completions deployment, a Responses deployment and a Foundry Claude Messages deployment through their documented clients. Check response content, tool-call identifiers, usage, cancellation and error behavior before replacing a custom transport. [4, 5, 9]

3. **Retain the execution controls you need.** Keep cross-model scheduling, concurrency limits, task/model logs and the scoreboard while comparing results. Agent Framework provides agent evaluation, but its inspected evaluation loop does not establish an equivalent to your comma-separated multi-model runner. [3, 15]

4. **Keep explicit model capabilities.** Record the route, supported tools, generation settings, context limits and timeout policy for each deployment. Native settings can remain in provider-specific options even when callers use IChatClient. Test the actual requested capability rather than treating a successful text response as proof of full compatibility. [1]

Choose where evaluations should execute

For the existing C# expenses workflow, the lowest-disruption choice is to keep the local runner and adopt Microsoft components selectively. For centrally managed model or agent evaluations, Foundry cloud evaluation can generate outputs from a configured target and score them. These are separate execution choices; using IChatClient does not require moving evaluation into the service. [2, 7]

A sensible acceptance test is parity on the same dataset and fixed judge, plus deterministic assertions for the taxi name and price. Measure request failures separately from answer quality. This is a proposed migration check, not a new benchmark result.

Limits, recent changes and evidence quality

Cloud evaluation has real target support

Current Foundry documentation describes model and agent targets under `azure_ai_target_completions`. The candidate deployment and evaluator's judge deployment are configured separately. Model router is now documented as an evaluation target, but cannot be selected for other evaluation model roles. Hosted-agent input formats differ between Responses and invocations protocols. [7]

Admin-connected models may serve as target, judge or conversation simulator. Foundry handles their configured endpoint and credentials, but the connected deployment must expose Chat Completions. Support is in preview and can vary by region. A native Messages-only endpoint does not satisfy that documented contract without a compatible intermediary. [11]

Local Python extensibility is a different mechanism

The Python `evaluate` API can run an application callable before scoring or score responses already present in data. That makes arbitrary applications evaluable, while leaving their protocol integration inside the callable. It does not itself prove an automatic Messages/Responses adapter. [8]

The public local judge configurations are Azure OpenAI and OpenAI configuration types. Current source also contains a BYO-model configuration, but explicitly describes it as internal service plumbing and intentionally unexported. Treat cloud connected-model support and the public local SDK contract separately. [13]

Two documentation inconsistencies matter

The Foundry SDK overview contains an outdated statement about the absence of a native Anthropic C# SDK. Current Anthropic SDK documentation and Microsoft's Agent Framework integration document that SDK and its IChatClient path. Use those current, concrete integration references. [6, 5, 9]

The Agent Framework evaluation guide shows a `FoundryEvals` constructor using `chatConfiguration`. Inspected Microsoft source instead takes a project client, judge deployment and evaluator specifications, and submits work through the project evaluation client. This confirms the integration but makes the installed package version decisive for exact code. [3, 16]

Scope and confidence

Research used Microsoft Learn, official Microsoft/Azure source, Anthropic's SDK documentation and the local bridge. Critical claims were checked in primary sources. Confidence is high that Microsoft supplies the shared .NET interface and evaluation integrations. No complete official matrix was found proving every Foundry catalog model and modality works through one evaluation target contract.

This is an architecture and documentation review as of 8 September 2026. Repository main branches are moving references. No packages were upgraded, deployments changed or model calls made for this research. Earlier expense results are context, not evidence of Microsoft adapter parity. The recommendation is to reduce custom transport code incrementally while retaining verified evaluation behavior.

Sources and implementation references

All web sources accessed 8 September 2026. Dates below are documentation update dates reported by the pages, not runtime compatibility guarantees. Numbered citations throughout the brief link to these primary sources.

[1] Microsoft Learn. Use the IChatClient interface. Updated 13 March 2026.

[2] Microsoft Learn. The Microsoft.Extensions.AI.Evaluation libraries. Updated 18 June 2026.

[3] Microsoft Learn. Agent Framework: Evaluation. Updated 25 August 2026; constructor example qualified in this report.

[4] dotnet/extensions. OpenAIClientExtensions.cs. Main-branch source; includes Chat and Responses adapters.

[5] Microsoft Learn. Agent Framework: Anthropic model provider. Updated 4 September 2026.

[6] Microsoft Learn. Microsoft Foundry SDKs and endpoints. Native SDK statement qualified in this report.

[7] Microsoft Learn. Evaluate model and agent targets with Microsoft Foundry SDK. Updated 7 September 2026.

[8] Microsoft Learn. azure.ai.evaluation package API. Current Python API reference.

[9] Anthropic. C# SDK. Official SDK, IChatClient integration and Foundry package guidance.

[10] microsoft/agent-framework. AnthropicClientExtensions.cs. Main-branch wrapper source.

[11] Microsoft Learn. Use admin-connected models in cloud evaluations. Updated 27 August 2026; preview.

[12] Microsoft Learn. IEvaluator.EvaluateAsync API. API reference; includes prerelease notice.

[13] Azure/azure-sdk-for-python. Evaluation model configurations. Main-branch source; public versus internal configuration boundary.

[14] Local project inspection. `InspectAzureAI.Maf/InspectChatClient.cs` and `InspectAzureAI.Maf/InspectAzureAI.Maf.csproj`, inspected 8 September 2026 in the working tree. Local paths are repository-relative for portability.

[15] microsoft/agent-framework. AgentEvaluationExtensions.cs. Main-branch evaluation orchestration source.

[16] microsoft/agent-framework. FoundryEvals.cs. Main-branch project evaluation integration source.